



DocShield

AI-Powered Document Fraud Detection System



Two-Engine
Detection



Under 30
Seconds



Zero-Trust Output



Explainable AI

Catch what a skim-read misses.

1. The Problem We're Solving

Every day, lenders, HR teams, and background-verification firms receive forged salary slips, fabricated Form 16s, and doctored bank statements. In India alone, document fraud costs the financial sector thousands of crores annually — and most of it sails through unnoticed because verification is still done manually, if at all.

The Core Pain Point

A PDF with a single font swap on a salary figure, a PAN that doesn't match its entity type, or a net pay that somehow exceeds gross income — these are easy mistakes to make in Photoshop and easy mistakes to miss in a rushed inbox. DocShield makes them impossible to miss.

What Fraudsters Actually Do

Attack Type	Example	How Often Missed Manually
Font swap on a single digit	₹50,000 → ₹5,00,000	Very often
Entity-type PAN mismatch	Company PAN on individual payslip	Almost always
Net > Gross salary	Net pay ₹3.5L on Gross ₹2L payslip	Frequently
Suspicious creator tool	Photoshop metadata on a bank statement	Always (no one checks)
Future-dated document	Payslip dated 2035	Occasionally
IFSC ↔ bank name clash	HDFC IFSC but listed under SBI	Always
Resubmitted tampered PDF	Same filename, altered content, different hash	Always

DocShield catches every one of these — automatically, in under 30 seconds, with a human-readable explanation for every flag.

2. Originality & Innovation

Most document-verification tools fall into one of two traps:

- They trust metadata blindly — ignoring that metadata is trivially editable.
- They run a black-box ML classifier that outputs a probability score with no explanation.

DocShield does neither. It introduces three original design choices:

2.1 Dual-Engine Independence

Forgery leaves traces in two unrelated places: **the file itself** (metadata, structure, how it was produced) and **what it claims** (numbers that don't add up, identities that don't match). A single model trying to catch both is mediocre at each. DocShield splits them into two completely independent engines that run in parallel — each going deep on its domain — and then combines their verdicts.

2.2 Explainability Over Probability

Every flag names exactly what is wrong — not '87% fraud probability' but 'gross_salary_mismatch: sum of earnings components (₹5,00,000) does not match stated gross salary (₹2,00,000)'. Each flag carries a severity level (CRITICAL / HIGH / MEDIUM / LOW) and a confidence score. The report is an audit trail, not a verdict.

2.3 Zero-Trust Output

The fraud-detection report is itself sensitive data. DocShield encrypts every annotated PDF with a freshly generated random password before it ever leaves the server — so a direct download from cloud storage is useless without that password. The original is in a private bucket; the annotated copy is in a separate private bucket. Both are namespaced per user.

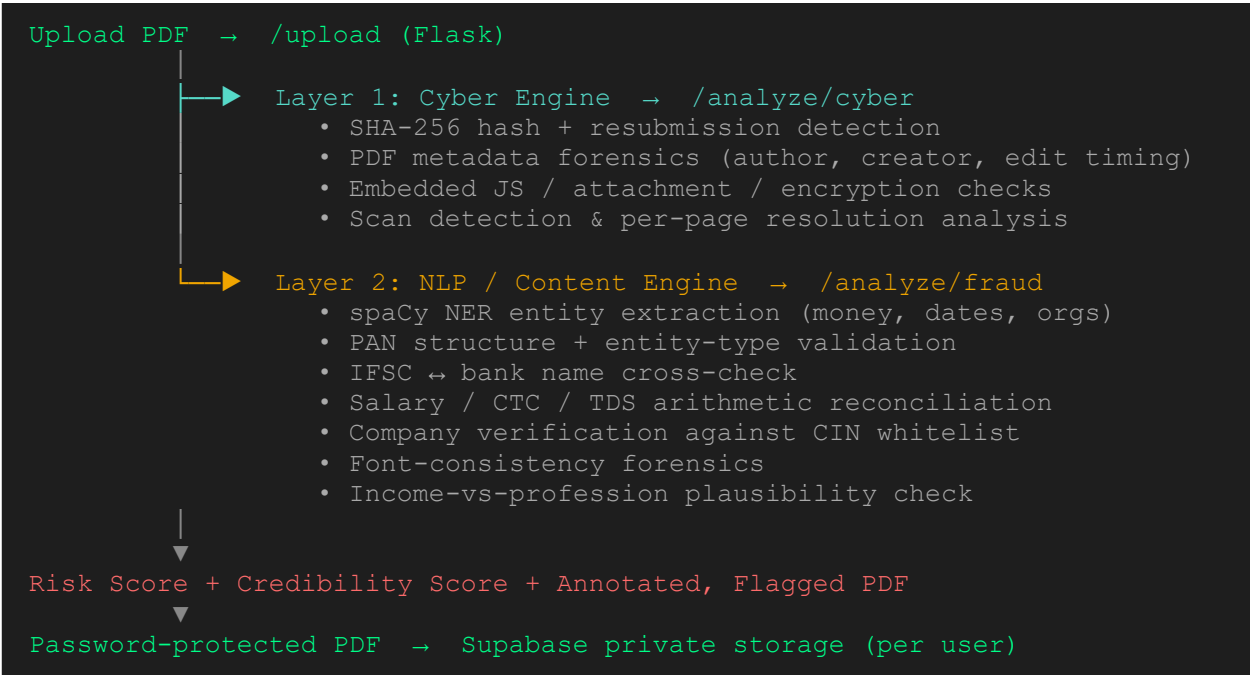
DocShield vs. Existing Solutions

Manual review: misses ~60% of forgeries. Black-box ML: explains nothing. Metadata-only tools: trivially bypassed. DocShield: two independent layers, every flag explained, output secured. No other open-source tool in this space combines forensic file analysis with NLP content analysis in a single, explainable, secured pipeline.

3. Technical Implementation

3.1 System Architecture

DocShield is a Flask web application with two independent detection engines that run sequentially (cyber first, fraud second, fraud result includes both in the final DB record).



3.2 File-by-File Breakdown

File	Role	Key Functions
start.py	Entry point	Dependency check → spaCy model check → launches Flask on :5000
app.py	Flask web server	/signup, /login, /logout, /me, /upload, /analyze/cyber, /analyze/fraud, /download, /report
cyber_detector_module.py	Layer 1 engine	check_resubmission(), check_metadata(), check_file_anomalies(), check_scanned_document(), run_cyber_analysis()
document_fraud_detector_enhanced.py	Layer 2 engine (core)	spaCy NER, PAN/IFSC/CIN validation, salary arithmetic, font forensics, PDF annotation via PyMuPDF
fraud_detector_module.py	Layer 2 wrapper	run_fraud_analysis() — thin adapter; moves annotated PDF to output folder, returns serializable JSON

config.py	Configuration hub	PROFESSION_SALARY_RANGES, COMPANY_WHITELIST, BANK_IFSC_MAP, RISK_WEIGHTS, SEVERITY_LEVELS — all tunable without touching engine code
hash_store.json	Resubmission memory	Per-path SHA-256 store; detects tampered resubmissions and cross-path duplicates
static/index.html	Analysis console	Upload UI, parallel results display, PDF.js viewer, annotated PDF + JSON report download
static/landing.html	Auth UI	Login / signup forms; cookie-based session; redirects to console on success

3.3 Tech Stack

Layer	Technology	Why This Choice
Backend	Flask (Python)	Lightweight, synchronous — perfect for sequential PDF processing pipelines
Cyber engine	PyMuPDF (fitz)	Low-level PDF struct access: metadata, encryption flags, embedded objects, per-page pixmaps for DPI estimation
NLP engine	spaCy en_core_web_sm	Fast, rule-augmented NER — extracts MONEY, DATE, ORG, PERSON entities without an LLM API call
PDF annotation	PyMuPDF	Writes color-coded flag boxes directly onto a copy of the source PDF
PDF encryption	pypdf PdfWriter	Encrypts annotated output before it leaves the server — output files are never stored unprotected
Auth	PBKDF2-SHA256 (100k iter)	Industry-standard password hashing; per-user salt; plaintext never stored or logged
Persistence	Supabase (Postgres)	users + documents tables; zero-config managed DB
File storage	Supabase Storage	Private buckets; per-user namespacing; secret-key-only access
Frontend	Vanilla HTML/CSS/JS	Zero build step; PDF.js for in-browser PDF rendering; dark 'security console' aesthetic

3.4 Scoring Model

Each engine produces its own score independently. The cyber score is a weighted sum of flag severities × confidence, normalized. The fraud (content) score uses the same weight table: **CRITICAL = 0.5, HIGH = 0.25, MEDIUM = 0.1, LOW = 0.05**. Final risk verdict: LOW / MEDIUM / HIGH / CRITICAL.

Score Range	Verdict	Meaning
≥ 0.70	CRITICAL	Clear signs of forgery — auto-reject recommended
0.50 – 0.69	HIGH	Multiple serious issues — mandatory human review

0.25 – 0.49	MEDIUM	Inconsistencies detected — verify before proceeding
< 0.25	LOW	Minor or no issues — document appears legitimate

4. Detection Coverage

4.1 Cyber / Forensics Layer

Check	Flag Type	Severity
SHA-256 hash comparison vs. stored submission	resubmission_detected	CRITICAL
Cross-path duplicate detection	duplicate_document	HIGH
Missing author metadata	author_missing	MEDIUM
Modification within 60 min of creation	rapid_modification	HIGH
Creator tool is Photoshop / GIMP / Canva / Inkscape	suspicious_creator_tool	HIGH
PDF is encrypted / password-protected	pdf_encrypted	HIGH
Embedded JavaScript detected	embedded_javascript	CRITICAL
Embedded file attachments	embedded_attachments	HIGH
Page count below expected minimum for doc type	suspicious_page_count	MEDIUM
File size under 5 KB	suspicious_file_size	MEDIUM
All pages are image-based (no selectable text)	fully_scanned_document	MEDIUM
Mix of native and scanned pages	mixed_document	MEDIUM
Per-page DPI below 150	low_resolution_scan	HIGH
DPI varies >100 across pages (spliced scans)	inconsistent_resolution	HIGH

4.2 NLP / Content Layer

Check	Flag Type	Severity
Gross salary $\neq$ sum of earnings components	gross_salary_mismatch	CRITICAL
Net salary > Gross salary	net_exceeds_gross	CRITICAL
Total deductions inconsistent	deductions_mismatch	HIGH
Annual TDS $\neq$ Monthly TDS $\times$ 12	annual_tds_mismatch	HIGH
Annual CTC vs Gross $\times$ 12 deviation > 20%	ctc_mismatch	HIGH
PAN fails regex pattern	invalid_pan_format	CRITICAL
PAN entity type wrong for document type (e.g. Company PAN on individual payslip)	pan_entity_type_mismatch	HIGH

Multiple different PAN numbers on same doc	multiple_pan_numbers	HIGH
IFSC prefix does not match stated bank name	ifsc_bank_mismatch	CRITICAL
Multiple different IFSC codes on same doc	multiple_ifsc_codes	MEDIUM
Company name not in CIN whitelist	unverified_company	MEDIUM
CIN format invalid	invalid_cin_format	HIGH
Date in the future	future_date	HIGH
Impossible calendar date (e.g. 32/13/2099)	invalid_date	CRITICAL
Salary wildly outside range for stated profession	salary_out_of_range	HIGH
Font inconsistency across document	font_inconsistency	MEDIUM

5. Security Design

Area	Implementation
Password storage	PBKDF2-SHA256, 100,000 iterations, unique random salt per user. Plaintext never stored or logged.
Session management	Server-side Flask sessions via signed cookies. Fixed FLASK_SECRET_KEY keeps sessions valid across restarts.
Output encryption	Every annotated PDF is encrypted with a freshly generated random password before upload to storage. Password shown once to user, stored in documents table.
File storage	Original + annotated PDFs in private Supabase Storage buckets; inaccessible without server's secret key. Per-user namespacing.
CORS	Locked to an explicit ALLOWED_ORIGIN (not wildcard) — required for credentialed cookie-based requests.
Auth endpoints	HMAC-safe compare_digest() for password comparison. /upload and /analyze routes return 401 if session missing.
Path traversal	/download sanitizes filename with os.path.basename() before serving from outputs folder.

6. Quick Start

Prerequisites

- Python 3.9+
- pip
- A free Supabase project (supabase.com)

Step 1 — Install dependencies

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

Step 2 — Configure environment

Create a .env file in the project root:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SECRET_KEY=sb_secret_XXXXXXXXXX
FLASK SECRET KEY=<a long random string - keep it consistent>
```

Step 3 — One-time Supabase setup

In the Supabase Dashboard → SQL Editor, paste and run supabase_setup.sql. This creates the users and documents tables.

Step 4 — Launch

```
python start.py
```

Then open <http://localhost:5000> — sign up, log in, and drop in a PDF.

7. Real-World Applicability & Scalability

Who Uses This

User Type	Use Case	Value
NBFC / Lending companies	Income document verification before loan disbursal	Catch salary fraud before funds are released
HR / Background verification firms	Employment certificate and payslip verification	Eliminate manual document review for routine checks
Banks	KYC document forensics at account opening	Automated first pass before compliance officer review
Insurance companies	Claim document verification	Detect fabricated medical bills or salary proofs
Government portals	Subsidy / scholarship application document check	Fair, consistent, scalable screening

Scalability Path

- `config.py` is already structured for per-organization whitelists (companies, banks, salary bands) — just populate the dicts and restart.
- The Supabase documents table stores every scan with full JSON results — a history / audit trail view is one query away.
- Batch upload is architecturally straightforward: `loop /upload + /analyze/cyber + /analyze/fraud` per file.
- Detection rules in `document_fraud_detector_enhanced.py` are modular functions — new fraud patterns can be added without touching the core engine.
- The dual-engine pattern scales to new document types (bank statements, ITRs, property valuations) by adding keyword sets to `config.py` and new validation functions.

Real Test Case ([sample.pdf](#) / [sample1.pdf](#) included)

The bundled sample PDFs are deliberately crafted with 16 planted fraud signals: future date, fake company spelling, invalid CIN, peon salary of ₹3.5L, net > gross, TDS mismatch, IFSC-bank mismatch, multiple conflicting IFSC codes, invalid PAN entity type, duplicate PANs, impossible dates, and OCR-style amount errors. DocShield flags all of them.

8. Project Structure

```
PaperTrail-AI/
├── start.py                # Entry point - dep checks + launch
├── app.py                  # Flask server: auth, routes, Supabase
├── config.py               # Tunable thresholds, whitelists, maps
├── cyber_detector_module.py # Layer 1: file/metadata forensics
├── cyber_detector.py       # Original cyber detector (unchanged)
├── document_fraud_detector_enhanced.py # Layer 2: NLP + PDF annotation
├── fraud_detector_module.py # Thin wrapper exposing Layer 2 to Flask
├── hash_store.json         # SHA-256 resubmission memory
├── requirements.txt
├── .env                    # Not committed - your secrets go here
├── supabase_setup.sql      # One-time DB schema
├── uploads/                # Temp storage for incoming PDFs
├── outputs/                # Annotated + encrypted PDF outputs
├── static/
│   ├── landing.html        # Login / signup UI
│   └── index.html          # Analysis console + PDF.js viewer
```

9. API Reference

Method	Endpoint	Auth?	Description
GET	/	No	Serves landing.html (login / signup)
POST	/signup	No	Create account — {email, password}
POST	/login	No	Authenticate — {email, password} → sets session cookie
POST	/logout	Yes	Clears session cookie
GET	/me	Yes	Returns {email} of logged-in user or 401
POST	/upload	Yes	Upload PDF file → {filename, size_kb, original_name}
POST	/analyze/cyber	Yes	Run Layer 1 on {filename} → cyber result JSON
POST	/analyze/fraud	Yes	Run Layer 2 on {filename} → fraud result JSON + annotated PDF path
GET	/download/<filename>	Yes	Stream annotated (encrypted) PDF
POST	/report	Yes	Generate + download full JSON report from {cyber, fraud, filename}

10. Roadmap

Already Architected, Not Yet Exposed

- Per-organization configurable whitelists via config.py — already structured for this.
- Audit trail / history view backed by the existing documents table.
- Batch upload for bulk verification workflows.

Next Phase

- Pluggable detection rules — new fraud patterns without touching the engine.
- Webhook integration — POST results to a lender's CRM on completion.
- Dockerized deployment — single docker-compose up for production.
- Multi-language NER support — Hindi, Tamil, Telugu document text.
- Admin dashboard — aggregate risk metrics across all users and documents.